



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: M.C Rene Adrian Davila Perez

Asignatura: Programación orientada a objetos

Grupo: 7

*No de
Práctica(s):* 6

Integrante(s): 323036285
323279866
323209432
323161891
323063609

*No. De Lista o
brigada* 3

Semestre: 2027-1

*Fecha de
entrega:* 28/ 09 / 2026

Obervaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	3
3. Desarrollo	4
4. Resultados	8
5. Conclusión	12

1. Introducción

Actualmente los estudiantes de diferentes áreas usan los modelos de lenguaje a gran escala para consultar conceptos técnicos específicos de programación, lo que genera un gran problema ya que muchos estudiantes usan la información sin verificarla y sin saber si es correcta o no, ya que estos modelos pueden confundir o mezclar conceptos entre distintos lenguajes de programación, afectando la calidad y confianza del aprendizaje.

Al mismo tiempo, al programar en java, al estudiantado se le pide tener una administración de los programas dentro de un package, lo cuál se vuelve un problema ya que no es solo acomodar los programas, hay que crear Java Docs, añadir manuales y hacer que queden bien acomodados usando comandos.

Resolver estos problemas nos ayudará a usar mejor a los LLM a la hora de consultar conceptos técnicos, consiguiendo una mejor comprensión y más veracidad en nuestro aprendizaje, también nos ayudará a tener mejor administración en nuestros programas y a darle mejor información al usuario acerca de cómo funciona.

Objetivos

- Investigar con apoyo de nuestro LLM buenas prácticas para el empleo de jars y javadocs, empleando un prompt eficiente para evitar que nos dé información inútil.
- Administrar los trabajos realizados anteriormente dentro de un package para tener una mejor administración y facilitar su entendimiento a diferentes usuarios.

2. Marco Teórico

Los paquetes en java [2] son utilizados para agrupar clases que se encuentren relacionadas entre sí, siendo capaces de también albergar documentos de texto, sub-clases, interfaces, etc, con el fin de hacer el código más sencillo de manejar, organizar y mantener.

Los paquetes hechos por el usuario [1] necesitan ser definidos en sus respectivas clases usando ‘package’ seguido de su dirección (unidas por puntos) junto a su nombre.

Los javadocs [3] se tratan de herramientas que generan APIs en formato HTML mediante java, siendo el estándar para documentar clases,

‘@author’ - Nombra a/los autores del documento

‘@version’ - Muestra la versión de la clase

‘@param’ - Informa los parámetros que se utilizan en un método

‘@return’ - Indica lo que retorna el método

‘@see’ - Asocia un método con otra clase

3. Desarrollo

Ejercicio 1

Para el primer ejercicio se le preguntó a nuestro LLM acerca de buenas prácticas para el uso de JAR y JavaDoc, y ¿Cuál es la mejor manera de generar un Javadoc? Esta fue la respuesta siguiendo con el prompt:

Soy estudiante de ingeniería y necesito que me expliques para el lenguaje de Java, buenas prácticas para el uso de JAR y JavaDoc , además, ¿cuál es la mejor manera de generar un JavaDoc?, no te salgas de la rama de ingeniería , explícame porque sería así , sé lo más claro posible.

1. Buenas prácticas para el uso de JAR (Java Archive)

Separar código fuente de binarios: Un JAR de distribución únicamente debe contener los archivos compilados (`.class`) y recursos (`.png`, `.properties`, etc.), jamás los archivos fuente (`.java`), archivos temporales de sistema ni carpetas de compilación intermedias.

Configuración explícita del Manifiesto (`MANIFEST.MF`): Si el JAR es ejecutable, el manifiesto debe definir con exactitud el atributo `Main-Class`.

Nomenclatura y versionado semántico: Nombrar el archivo bajo una convención estandarizada: `nombre-proyecto-vX.Y.Z.jar`.

Estructura interna de paquetes: El JAR debe respetar la jerarquía de directorios exacta declarada en el `package`.

Modularidad y dependencias: No empaquetar librerías externas completas dentro del archivo salvo que se busque expresamente construir un *fat JAR*; de lo contrario, las dependencias deben gestionarse externamente mediante el `Class-Path`.

2. Buenas prácticas para el uso de Javadoc

Documentar el "qué" el "por qué", no el "cómo".

Uso riguroso de etiquetas estándar:

`@param` para cada argumento que recibe un método o constructor.

`@return` para describir el valor devuelto.

`@throws` o `@exception` para documentar cualquier excepción lanzada.

`@author` y `@version` obligatorios.

Primera oración concisa: debe terminar en punto y sintetizar claramente la

función.

Mantener sincronía con el código: actualizar el bloque Javadoc si cambia la firma del método.

3. ¿Cuál es la mejor manera de generar un Javadoc?

A. A nivel profesional / industrial: Herramientas de construcción (Maven o Gradle). Ejemplo: `mvn javadoc:javadoc` o `./gradlew javadoc`.

B. A nivel académico / consola pura: Comando directo del JDK. Ejemplo: `javadoc -d doc -sourcepath src -subpackages mx -encoding UTF-8 -charset UTF-8`.

C. En el IDE (VS Code / IntelliJ / Eclipse): Generación automática escribiendo `/**` y presionando Enter sobre la clase o método.

Ejercicio 2

Para el segundo ejercicio se creó un paquete general que reúne las prácticas y proyectos realizados durante el curso. Cada práctica, así como la primera etapa del proyecto, se organizó con la siguiente estructura:

- mx
- unam
- die
- poo
- g7
 - practica 1
 - src
 - test
 - lib
 - doc
 - practica_1.jar
 - practica 2...
 - etapa_1

Generación de archivos JAR

Para empaquetar cada práctica se utilizó el comando `jar` desde la terminal, con la siguiente sintaxis:

```
jar -cvfe nombre.jar ClasePrincipal -C carpeta.clases .
```

Generación de archivos JAR

Como ejemplo la práctica 1: `jar -cvfe practica_1.jar P1 -C test .`

`cvfe` viene de:

`c` - crear

`v` - verbose

`f` - archivo de salida

`e` - entry point

Comprime las clases compiladas (`.class`) que están en la carpeta `test` y las mete dentro de un archivo `.jar`, y aquí usamos el `.e` de `cvfe` para establecer la clase principal que tiene el `main` (punto de entrada), anotándola en el archivo interno `MANIFEST.MF` para que el archivo sea ejecutable con un solo comando.

Siguiendo con el ejemplo de la práctica 1:

```
jar tf practica_1.jar
```

Lista en la terminal todo lo que está comprimido dentro del `.jar` (incluyendo la carpeta `META-INF/MANIFEST.MF` y los `.class`), sirviendo como prueba de que el paquete se armó correctamente sin tener que descomprimirlo.

```
java -jar practica_1.jar
```

Arranca y corre el programa de Java de manera autónoma directamente desde el archivo `.jar`, sin necesitar el código fuente `.java` ni invocar clases individuales.

Generación de la documentación con Javadoc

Además del archivo `.jar`, cada práctica contenía el `src` (el código fuente `.java`), el `test` (el código `.class`) y el `doc` (la documentación de cada clase y métodos en HTML). Para este último se utilizó la siguiente sintaxis:

```
javadoc -d direccion_del_doc direccion_del_paquete
```

que usamos como:

```
javadoc -author -version -d doc src/*.java
```

Lee automáticamente los comentarios `/** ... */` del código fuente (autores, versión, `@param`, `@return`) y construye todo el sitio web HTML interactivo con la documentación de las clases y métodos.

Para la primera etapa del proyecto fue similar con las prácticas, sin embargo en el proyecto existían 5 clases: `Main.java`, `Sistema.java`, `Paciente.java`, `Medico.java` y `Enfermero.java`; así que por cada clase se generaron todos los archivos con la información proporcionada anteriormente.

4. Resultados

Al abrir la documentación de la etapa 1, este fue el resultado que nos arrojó mostrando la API completa del sistema hospitalario con sus clases, autores y métodos:



The screenshot shows the Javadoc documentation for the package `mx.unam.die.poo.g7.etapa_1.src`. The page has a navigation bar with links: [Overview](#), [Tree](#), [Index](#), [Search](#), and [Help](#). Below the navigation bar, there is a search box and a list of classes and interfaces. The main content area displays the package name and a list of classes with their descriptions.

Class	Description
<code>Enfermero</code>	Clase que se usa para generar la información del enfermero, sus atributos y sus comportamientos como registrar en el sistema, dar tratamiento y ver la lista de pacientes
<code>Main</code>	Clase principal para comunicarse con las demás clases y ejecutar el flujo del sistema hospitalario
<code>Medico</code>	Clase para generar un médico con sus atributos como nombre, cédula y especialidad y sus comportamientos como registro en el sistema, dar consulta, dar tratamiento y ver pacientes
<code>Paciente</code>	Permite conocer y manipular la información del paciente, sus datos personales y su estado dentro del proceso de atención médica
<code>Sistema</code>	Gestiona el sistema hospitalario general, el registro de médicos, enfermeros y pacientes, así como la asignación de pacientes a los profesionales de la salud y el archivo de curados

En la documentación de la Práctica 4 se ve la descripción de la clase `AnalizadorPalabras` y sus funciones con las etiquetas `@param` y `@return`:

Class `AnalizadorPalabras`

`java.lang.Object`
`mx.unam.die.poo.g7.practica_4.src.AnalizadorPalabras`

`public class AnalizadorPalabras`
`extends Object`

La clase `AnalizadorPalabras` se encarga de los métodos para limpiar y contar las palabras de la oración, identificar cuántas de ellas son distintas y determinar cuántas veces se repite cada una

Field Summary

Fields		
Modifier and Type	Field	Description
<code>Map<String, Integer></code>	<code>frecuencias</code>	Mapa que almacena cada palabra y su frecuencia de aparición
<code>String</code>	<code>oracion</code>	Cadena de texto que contiene la oración a analizar

Constructor Summary

Constructors

Constructor	Description
<code>AnalizadorPalabras(String# nuevaOracion)</code>	Constructor que inicializa la oración a analizar y crea la estructura HashMap en memoria

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
void	<code>contarPalabras()</code>	Verifica que haya oración, la pasa a minúsculas y quita caracteres especiales para evitar errores, crea un arreglo de cadenas de texto con las palabras de la oración y registra la frecuencia de cada una en el mapa
void	<code>mostrarDuplicadas()</code>	Muestra en consola en orden alfabético las palabras duplicadas junto con su frecuencia de aparición
int	<code>obtenerNumeroDuplicadas()</code>	Recorre las llaves del mapa y cuenta cuántas palabras aparecen más de una vez

Methods inherited from class Object

`clone()`, `equals()`, `finalize()`, `getClass()`, `hashCode()`, `notify()`, `notifyAll()`, `toString()`, `wait()`, `wait()`, `wait()`

Field Details

oracion

```
public String# oracion
```

Cadena de texto que contiene la oración a analizar

frecuencias

```
public Map#<String#,Integer#> frecuencias
```

Mapa que almacena cada palabra y su frecuencia de aparición

Constructor Details

AnalizadorPalabras

```
public AnalizadorPalabras(String# nuevaOracion)
```

Constructor que inicializa la oración a analizar y crea la estructura HashMap en memoria

Parameters:

`nuevaOracion` - Cadena de texto a analizar

Method Details

contarPalabras

```
public void contarPalabras()
```

Verifica que haya oración, la pasa a minúsculas y quita caracteres especiales para evitar errores, crea un arreglo de cadenas de texto con las palabras de la oración y registra la frecuencia de cada una en el mapa

obtenerNumeroDuplicadas

```
public int obtenerNumeroDuplicadas()
```

Recorre las llaves del mapa y cuenta cuántas palabras aparecen más de una vez

Returns:

retorna la cantidad de palabras diferentes que se encuentran duplicadas

mostrarDuplicadas

```
public void mostrarDuplicadas()
```

Muestra en consola en orden alfabético las palabras duplicadas junto con su frecuencia de aparición

Se generaron los archivos ejecutables JAR para cada una de las prácticas dentro de sus directorios:

```
FullName
-----
C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\etapa_1\etapa_1.jar
C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\practica_1\practica_1.jar
C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\practica_2\practica_2.jar
C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\practica_3\practica_3.jar
C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\practica_4\practica_4.jar
C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\practica_5\practica_5.jar
```

Mediante la herramienta `jar tf` se comprobó la estructura del archivo `etapa_1.jar`, y las clases que tenía `.class`:

```
META-INF/
META-INF/MANIFEST.MF
Enfermero.class
Main.class
Medico.class
Paciente.class
Sistema.class
```

Al ejecutar el archivo `practica_1.jar` directamente desde la consola, el programa de la calculadora inició:

```
PS C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\practica_1> java -jar practica_1.jar
Calculadora
Ingresa el primer número:
3
Ingresa el segundo número:
4
Seleccione la operacion:
1- Suma
2- Resta
3- Multiplicacion
4- División
5- Potencia
6- Raíz
7- Módulo
1
Suma:
El resultado de la suma es :
7.0
```

Ejecutamos el archivo `practica_3.jar` comprobando si un número es palíndromo por consola:

```
PS C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\practica_3> java -jar practica_3.jar
Programa para determinar si un número de 5 dígitos es un palidromo o no.
Favor de ingresar un número entero positivo de 5 dígitos: 12321
12321 es un palíndromo.
PS C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\practica_3>
```

Se ejecutó practica_4.jar introduciendo una cadena de texto y obteniendo el conteo y desglose de palabras duplicadas en consola:

```
PS C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\practica_4> java -jar practica_4.jar
Ingresa la oración a analizar:
hola mundo hola java mundo hola

Total de palabras diferentes que se duplicaron: 2
Palabras duplicadas:
hola: 3
mundo: 2
```

Finalmente, se ejecutó etapa_1.jar demostrando el flujo completo del sistema hospitalario y la gestión de pacientes y médicos:

```
PS C:\Users\German\Desktop\C\mx-corregido2\unam\die\poo\g7\etapa_1> java -jar etapa_1.jar
Registro de médicos:
¿Cuántos médicos vas a registrar? 1

Datos del Médico 1:
Nombre: Juan
Especialidad: cardio
Cédula: 1234

Registro de enfermeros:
¿Cuántos enfermeros vas a registrar? 1

Datos del Enfermero 1:
Nombre: Romina
Especialidad: cardio
Cédula: 4321

Registro de pacientes:
¿Cuántos pacientes vas a registrar? 1

Datos del Paciente 1:
Nombre: Estefano
Apellidos: Roblez
Especialidad que requiere: cardio
paciente registrado

-----
Hospital: Hospital La Raza

Asignación a médicos:
Paciente Estefano asignado al médico Juan

Asignación médica:
Lista de pacientes ordenada del médico :Juan
Roblez Estefano - Especialidad: cardio (0)
El nombre del paciente es: Estefano Roblez y su tratamiento actual es: En consulta
El médico indicó tratamiento a: Estefano

Asignación a enfermeros:
Paciente Estefano asignado al enfermero Romina

Tratamiento de enfermeros:
Lista de pacientes ordenada del enfermero :Romina
Roblez Estefano - Especialidad: cardio (0)
Dando tratamiento a: Estefano
El nombre del paciente es: Estefano Roblez y su tratamiento actual es: Curado
Paciente Estefano ahora está curado
```

5. Conclusión

Se cumplieron los objetivos de esta práctica. En primer lugar, investigamos con ayuda de nuestro LLM las buenas prácticas para el uso de JARs y Javadocs, empleando un prompt claro y específico para evitar información innecesaria. Además, contrastamos sus respuestas con la documentación oficial, lo que nos ayudó a entender que los LLM son una herramienta de apoyo, pero que su información debe verificarse antes de usarla.

En segundo lugar, logramos organizar los trabajos de prácticas anteriores dentro de un package, con una estructura ordenada de carpetas (src, lib, test y doc). Generamos la documentación con Javadoc, incluimos los manuales y empaquetamos el programa en un archivo JAR mediante comandos desde la terminal. Las pruebas confirmaron que el paquete se compila y ejecuta correctamente, y que la documentación explica de forma clara cómo funciona cada clase, por lo que otros usuarios pueden entender y usar nuestros programas con facilidad. De esta manera se completaron satisfactoriamente los ejercicios que se dejaron.

Referencias

- [1] GeeksforGeeks. *User-defined Packages in Java*. <https://www.geeksforgeeks.org/java/user-defined-packages-in-java/>. Publicado: 2026.
- [2] Programarya. *Paquetes en Java*. <https://www.programarya.com/Cursos/Java/Paquetes>. Publicado: 2026.
- [3] Wikipedia. *Javadoc*. <https://es.wikipedia.org/wiki/Javadoc>. Publicado: 2026.